Tutorial 25 — LLM-Driven Evolutionary Search

A bounded signed Max-Cut classroom demonstration

ENGR 689 Tutorial Team

October 28, 2026

Contents

1 Evolutionary optimization **2**
1.1 Optimization as search . . . 2
1.2 Population and fitness . . . 2
1.3 Variation, evaluation and selection . . . 3
1.4 Exploration and exploitation . . . 4

2 A language model as the mutation operator **4**
2.1 The language-model mutator . . . 4
2.2 Signed Max-Cut . . . 5
2.3 Local moves and feasible witnesses . . . 6

3 Building and observing the system **7**
3.1 Solver programs . . . 7
3.2 Four numerical algorithms . . . 7
3.3 Initial population and fair starts . . . 8
3.4 Designing the three experiments . . . 8
3.5 Demo 1: a limited initialization . . . 9
3.6 Demo 2: a researcher-selected suite . . . 9
3.7 Comparing the evidence . . . 10
3.8 Demo 3: a dense real-weight stress case . . . 10
3.9 Reading solver evidence . . . 11

4 Formal definition and transfer **11**
4.1 The agentic evolutionary framework . . . 11
4.2 Applying the framework to research . . . 12

A Appendix: architecture and API **13**
A.1 Server architecture . . . 13
A.2 The API request and context . . . 13
A.3 Response and loop closure . . . 14

References **14**

1 Evolutionary optimization

1.1 Optimization as search

When the best candidate is unknown, what can we do?
Define a candidate space $\mathcal{A}$ and a measurable objective F. Seek

$$a^* \in \arg\max_{a \in \mathcal{A}} F(a).$$

Generate candidates → measure them → spend the next trial where the evidence suggests it may help.
Optimization is an evidence-guided search. An evolutionary algorithm adds a population, inherited variation and selection.

The candidate may be a parameter vector, a program or a design. An optimizer does not need to enumerate the whole space, but it needs an operational definition of quality. A candidate that sounds plausible is not a measured result. Our objective can be computed automatically, which lets repeated search use the same external check.

Begin with evolutionary computation itself. Language models are introduced later as one possible variation mechanism. This ordering separates the optimization framework from the technology used to propose a change. It also makes a numerical-only baseline meaningful: the framework does not require a language model to define or evaluate a candidate.

1.2 Population and fitness

Why keep several candidates instead of only the current champion?
Fitness measures how well a candidate performs now. A different candidate may offer changes that the champion's structure does not readily suggest.
Hypothetical example, equal mutation budgets:
A: current fitness **7** → best tested child **8**.
B: current fitness **6** → best tested child **9**.
Keep promising alternatives because a lower-scoring parent can lead to a better descendant. Multiple parents preserve several paths for future search.

Pause after the question and let students argue for keeping only the champion: it has the highest measured fitness, so why spend effort elsewhere? The missing information is how good the candidates' descendants could be. Current fitness evaluates a candidate under a declared protocol; it does not evaluate every possible mutation of that candidate or of its competitors.

The numbers are a hypothetical maximization example, not Max-Cut measurements or predictions. Suppose A and B have different structures, and we give each the same mutation and evaluation budget. A currently scores 7 and its best tested child scores 8; B currently scores 6 and its best tested child scores 9. Keeping only A would discard B as a direct starting point before discovering that child. B's lower current score therefore does not make it useless for future search. The mechanism is that inherited structure affects which changes are available or likely under the variation operator. A search from A might eventually rediscover B's structure; retaining B keeps that route directly available.

A population reduces dependence on descendants of a single parent, but it costs evaluations and does not guarantee improvement. Several nearly identical programs may offer little useful variety. The

benefit depends on their behavioral differences, the variation operator, selection and the available budget. This example motivates retaining promising alternatives; it does not justify retaining every weak candidate.

In the classroom controller the population contains solver programs, with a cap of three complete evaluated survivors and deterministic fitness ranking. Up to three survivors enter Luna's context as potential parents; a later execution of a solver produces a separate solution witness. There is no reserved diversity slot, island migration or tournament sampling. The controller preserves highly ranked alternatives when available, but it does not explicitly enforce structural or behavioral diversity, and a potentially useful lower-ranked program can still be discarded.

1.3 Variation, evaluation and selection

What turns repeated candidate generation into an evolutionary algorithm?

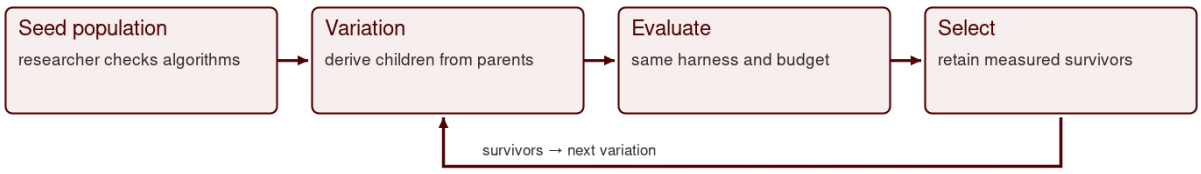


Figure 1: Seed once; derive children from survivors; evaluate and select again.

Mutation can perturb a parameter, replace a component or compose stages. Selection compares parents and children using the same fitness protocol.
Useful structure is inherited, varied and tested. Elitism retains measured parents when a mutation is worse.

A classical variation operator can make random parameter edits, replace a subtree in a program or combine material from parents. Variation does not have to improve fitness. The evaluator supplies evidence and selection decides which structures remain. Carrying parents into the enlarged pool prevents an unsuccessful child from automatically erasing the incumbent.

Chapter 10 distinguishes initial generation from survivor-conditioned mutation. Its recurrence selects parents, generates variants, pools parents and children, then selects again. Here the controller retains prior measured scores, so elitism protects the observed leaderboard under the fixed protocol. That is not a theorem about expected quality on new trials or eventual global convergence.

1.4 Exploration and exploitation

Should the next mutation refine a strong candidate or try a different mechanism?
Exploit: adjust a promising parameter or refine an existing composition. **Explore:** replace a stage or try another algorithm family.
Too little variation can collapse the population. Too much unconstrained variation can waste evaluations on poor candidates.
Variation and selection must balance inheritance with discovery. Better starting material may help; a weak start does not imply permanent failure.

The search neighborhood is defined by the representation and mutation rules. An operator limited to small parameter changes explores a different neighborhood from one allowed to replace a kernel. Strong selection can remove useful alternatives; weak selection can spend resources on low-quality programs. The controller exposes complete-trial ranking and exact-duplicate rejection but does not implement a general diversity guarantee.

The distinction matters for the initialization experiment. Checkboxes choose the initial population only. They do not remove unchecked kernels from the artifact space. A language-model mutator may therefore recover a useful algorithm absent from the initial suite. Claiming that a poor suite can never reach the reference would contradict that permitted search space.

2 A language model as the mutation operator

2.1 The language-model mutator

What changes if the variation operator can read programs and measured feedback?

1. **Retain vector optimization:** adjust one vector per vertex to improve a continuous cut objective.
2. **Round:** convert the optimized vectors into a two-group partition.
3. **Luna proposes an extension:** append local search, starting from that partition.
4. **Evaluate the child:** compare it with the parent using identical trials and the same total time limit per trial.

The child inherits vector optimization and adds local search. Whether the extension helps is a hypothesis tested by numerical evaluation.

Present the numbered example as a hypothetical mutation, not a claim about a particular recorded Luna response. The candidate being changed is a solver program. Its execution produces a partition, which the evaluator uses to measure the program's quality.

Step 1 retains the parent's vector method. Instead of immediately choosing one of two groups for each vertex, the Burer–Monteiro method optimizes continuously adjustable vectors using an objective related to the cut value. Our implementation uses two-dimensional unit vectors. This is a numerical search over a continuous surrogate; it does not certify the global Max-Cut optimum.

Step 2 rounds the vectors into a feasible partition. A direction through the vector space determines which side each vector falls on, assigning each graph vertex to one of two groups. Vector optimization and rounding are internal operations of the existing Burer–Monteiro kernel, not two independently editable strategy stages.

Step 3 is the proposed program mutation. Luna reads selected parent programs, measured trial results and the allowed edits, then may propose appending a trusted local-search kernel. That

kernel receives the partition produced by the first stage and explores changes such as moving vertices between groups. The parent recipe is vector optimization followed by rounding. The child recipe retains both operations and adds local search. The numerical kernels execute these operations after the proposal has passed validation.

Step 4 tests the complete child program against the parent under the declared evaluation protocol: the same graph, seeds, raw starting partitions and numerical limits. Each program has the same total time allowance per trial; the child shares its allowance across its stages. Appending local search does not grant extra evaluation time. The evaluator independently rescores the original graph and aggregates complete trials into fitness. The added stage might help, offer no improvement, or reduce performance within the fixed allowance. The proposal is a hypothesis because those outcomes must be measured rather than inferred from a plausible explanation.

The model replaces one part of the evolutionary algorithm: the variation operator. It does not replace the artifact representation, objective, evaluator, selection rule or external population memory. Between calls, the controller changes the context; it does not update model weights. The same selected structure and failure record can make the next proposal more informed than an independent guess.

Chapter 10 connects context engineering to mutation and evaluation to fitness. FunSearch is a primary program-search example (Romera-Paredes et al. 2024). The classroom system uses a narrower interpreter over existing numerical kernels. Its proposals can change configurations and compositions but cannot invent arbitrary executable Python or modify the objective. Better proposals are possible, never guaranteed.

2.2 Signed Max-Cut

Can every edge preference be satisfied at once?
Illustrative triangle: **A–B = +3; A–C = −2; B–C = −2.**
Positive weights reward separation. Negative weights penalize separation.

Partition	Crossing contributions	Cut value
A, B, C together	None	0
A alone; B with C	$+3-2$	**1**
C alone; A with B	$-2-2$	-4

Edge preferences can conflict. Maximize their total crossing weight. Separating A and B forces at least one negative edge to cross.

The purpose of this frame is to establish a conflict that requires a global tradeoff before introducing the objective's notation. This three-vertex triangle is a hypothetical teaching example, not an instance measurement from the live demonstration. There is an edge between every pair: A–B has weight +3, and A–C and B–C each have weight −2. A partition assigns every vertex to one of two groups; a group may be empty.

Show the question, pause, and invite students to try satisfying all three preferences. The positive edge asks for A and B to be separated. Both negative edges ask for C to remain with their respective endpoints. C cannot be in the same group as both A and B while those two vertices are separated. Individual edge preferences therefore cannot all be satisfied at once.

Reveal the table and compute each row aloud. Keeping all three vertices together cuts no edges, giving 0. Separating A while keeping B and C together cuts A–B and A–C: +3 minus 2 equals 1. Separating C while keeping A and B together cuts both negative edges: −2 minus 2 equals −4. Separating B gives the symmetric score 1. Together with exchanging the two group names, these exhaust the possible partitions, so 1 is the optimum for this small illustrative triangle. That direct enumeration does not certify either live benchmark's reference value.

Conclude: a solver must account for the whole graph because a single vertex's placement affects several edges. A crossing contributes its signed weight; an uncut edge contributes zero. Keeping a negative edge uncut avoids a penalty rather than earning an extra positive reward. Maximizing the total crossing weight balances the incompatible preferences.

For the formal definition, encode the two group memberships by $s_i \in \{-1, +1\}$. Equal labels have product $s_i s_j = 1$, so the crossing indicator $(1 - s_i s_j)/2$ is 0. Opposite labels have product -1, so the indicator is 1. Thus

$$C_G(s) = \sum_{(i,j)\in E} w_{ij} \frac{1 - s_i s_j}{2}, \qquad \max_{s\in\{-1,+1\}^{|V|}} C_G(s).$$

Complementing every sign exchanges the group names without changing the cut value. The demonstration evaluates each solver's returned partition by independently summing the original graph's edge contributions. This gives external numerical feedback for evolving solver programs. Graph size, trial conditions and unproven reference values are introduced with the experiment setup, after students understand what a score means.

2.3 Local moves and feasible witnesses

Can the search move to a worse state while keeping a better result?
Flip one sign, $s_i \leftarrow -s_i$, with exact gain

$$\Delta_i = s_i \sum_{j:(i,j)\in E} w_{ij} s_j.$$

The exploratory assignment may worsen. A separate best-so-far copy preserves the strongest valid witness seen during that execution.
Separate the moving state, best witness and strategy fitness. Only a complete evaluation makes a strategy eligible for ranking.

Only terms incident to the flipped vertex change; their differences sum to the stated gain. Cached gains make local moves inexpensive. Numerical methods can accept neutral or worsening moves to escape a basin while retaining the best feasible assignment separately.

This distinguishes three levels. The strategy is a program. Executing it produces a sign vector. The evaluator recomputes that vector's objective and summarizes three trials into strategy fitness. A witness from a censored trial may be valid even though the program is ineligible for complete-trial selection. A zero-cut partition is legal; a vector of zero-valued signs is not.

3 Building and observing the system

3.1 Solver programs

What can Luna edit if the assignment itself is not the evolving artifact?
A program has one to three stages: algorithm, configuration, work cap and start rule.
`bm(max_sweeps=20, rounding_count=8); retained`
`→ bls(max_work=9000); retained`
Execute a program → obtain signs → recompute the original cut → summarize three trials → rank the program.
Evolve the solver; evaluate its witness. Trusted kernels execute the stages. Unchecked algorithms remain available to the mutator.

The identifiers bm and bls denote the rank-two Burer–Monteiro method and Breakout Local Search. The displayed composition was proposed in the earlier genuine dense-graph rehearsal. Each schema stage contains backend, config, max_work and fresh. Missing defaults are normalized before hashing so equivalent encodings cannot receive separate evaluations merely by omitting a default.

A retained stage receives a copy of the best witness from preceding work in the same trial. A fresh stage uses a seeded random exploratory start; the interpreter retains an older better witness if the new stage is worse. Remaining numerical time is allocated among remaining stages, with unused time carried forward. The model supplies data describing this bounded interpreter, not shell commands or arbitrary source code.

3.2 Four numerical algorithms

Which mechanism changes when we choose a different solver stage?
Tabu Search: best admissible flips, temporary reversal bans and aspiration. **Simulated Annealing:** temperature-dependent acceptance of random flips.
Breakout Local Search: improving descent followed by adaptive perturbations. **Rank-two Burer–Monteiro:** unit-vector refinement, projection to signs and polish.
Acceptance, memory, escape and representation are different design levers. Full examples and parameter meanings are available in the algorithm guide.

Tabu Search maintains current signs, one-flip gains, tabu expiry and a separate best witness. It flips the highest-gain admissible vertex even when the gain is nonpositive. Seeded ties are reproducible. A flip at iteration t sets expiry to t+tenure+1; aspiration admits an otherwise prohibited flip if it would beat the global incumbent. restart_after=0 disables a stagnation restart. The work cap is 20000 iterations.

Simulated Annealing proposes a uniformly selected vertex and accepts a gain Δ with probability $\min(1,\exp(\Delta/T))$. Temperature falls geometrically over the proposal budget. A logarithmic comparison avoids unstable exponentiation; optional bounded improving-flip polish shares the trial clock. The cap is 200000 proposals.

Breakout Local Search alternates steepest improving descent and perturbation. Repeating the same local optimum can increase jump magnitude. Its adaptive mechanism chooses directed tabu single/pair moves or random perturbation. The controls jump, stagnation, p_min, q and tenure_min/max govern that choice; the cap is 12000 individual flips. These work units are not interchangeable with tabu iterations or annealing proposals.

The Burer–Monteiro-style implementation uses two-dimensional unit vectors, coordinate refinement, projection directions and discrete polish. It allows one solve with at most 60 sweeps and 16 directions. An encountered nonconvex relaxed value is not a certified Max-Cut upper bound. Warm signs participate in the preserved extended path and remain eligible for retention. All 133 hashed reference files stay unchanged; classroom adaptations live outside the vendor snapshot.

3.3 Initial population and fair starts

Do the solver checkboxes change the initial population or the whole search space?
Checkboxes choose P_0. Every selected program and proposed child receives copies of the same raw starts for seeds 3, 4 and 5.
All four trusted kernels remain available to Luna. Both current graphs allow 8 numerical seconds per trial. The selected seed programs produce distinct initial fitness on the 1,000-vertex graph.
Change initialization deliberately and hold the comparison protocol fixed. A missing seed algorithm can still appear in a later mutation.

The first two scenarios hold the 1,000-vertex signed graph, raw starts, seeds, per-trial allowance, kernel catalog, selection and model-call cap constant. Only the seed suite differs. A numerical screen with the same initial programs measured medians 921 for Simulated Annealing, 975 for Tabu Search, 992 for Breakout Local Search and 1012 for Burer–Monteiro. The reference is 1098. These are program-and-budget measurements, not universal rankings of algorithm families. The limited suite contains Tabu Search and Simulated Annealing. The informed suite contains Tabu Search, Breakout Local Search and rank-two Burer–Monteiro. Initial configurations for a given kernel are identical across scenarios; no hidden weak variant is substituted.

The initial configurations are disclosed: tabu tenure 5 with 800 iterations; annealing temperature 2 to 0.01, 6000 proposals and polish; Breakout Local Search 700 flips; Burer–Monteiro 20 sweeps and 8 directions. More seed programs mean more initial evaluations, so numerical effort must be reported alongside calls and wall time. Researcher design effort before the run is not measured by the run clock.

3.4 Designing the three experiments

Which differences can our initialization comparison actually explain?
Demos 1 and 2: same signed graph, seeds, trial allowance, mutation catalog and call cap. Change only the initial solver suite.
Report initial median, calls and time to the median target, all-trial robustness, numerical effort and total teaching-run time.
Declare the success criterion before comparing runs. The third graph is a separate dense real-weight stress case.

The first two experiments use the same 1,000-vertex signed graph with 5,043 edges and supplied reference 1098. The third uses a separate 100-vertex dense graph with 4,950 real-valued signed edges and supplied reference 402.5209. Neither reference is a certified optimum. Earlier torus records remain historical evidence rather than substitutes for these cases.

Primary fitness is median cut across three complete trials. Matching the reference median is weaker than matching it in every trial; the latter additionally requires the minimum to meet the reference. The comparison page exposes both milestones. It reports calls and time to target separately from all calls spent in a fixed three-round teaching run.

These are bounded classroom observations with a stochastic model, not repeated research estimates. Larger seed suites consume more initial numerical effort. Human prior knowledge and its development cost are not captured by the run clock. Changing graph structure and weight distribution in demo three prevents a causal speed comparison with the first two; it tests whether the same architecture still operates on a denser signed problem. Three calls are permitted by the server; the replacement rehearsal has two successful-call slots per case after two network-restricted attempts, all counted within the authorized allowance.

3.5 Demo 1: a limited initialization

Can local-search seeds recover a useful relaxation method through mutation?
Use the **1,000-vertex signed graph (5,043 edges)**, seed 3, **Tabu Search and Simulated Annealing**. The initial best median is 975; reference 1098 is unproven.
All four kernels remain available. Inspect parent programs, the actual request, public proposals and complete measured child fitness.
A limited suite defines a starting point, not a permanent restriction. Follow improvement, reference gap, numerical effort and model calls.

Use the first named preset and confirm two checked algorithms. The server verifies graph, seed and selection metadata. Start evaluates only those programs. At each gate inspect the selected parents and explicitly send their context. A two-parent population supplies two actual parents; no manufactured copies are added to make three.

The earlier BM-only torus demonstration was an algorithm-recovery control, not a recommendation to initialize every task with Burer–Monteiro. That easy graph saturated near its reference and obscured differences. The replacement screen uses unchanged seed configurations and identical raw seeds. Burer–Monteiro is actually the strongest initial program here, so excluding it from this limited suite tests whether mutation can introduce a useful absent mechanism. Recovery and reference matching are hypotheses, never scripted outcomes. The genuine replacement run completed two Luna rounds in 24.003 seconds. Its eligible median improved from 975 to 1008; its best witness was 1031. No reference match occurred. Luna refined the two existing mechanisms rather than introducing Burer–Monteiro, even though all kernels remained available.

3.6 Demo 2: a researcher-selected suite

What changes when measured domain knowledge supplies stronger starting programs?
Keep graph, seeds and caps fixed. Select **Tabu Search, Breakout Local Search and rank-two Burer–Monteiro**.
The initial best median is 1012, compared with 975 for the limited suite. The reference gap falls from 123 to 86 before any model call.
Researcher knowledge improves this starting population. Evaluate later gains and costs; a stronger start does not guarantee a faster run.

The initial medians come from complete numerical trials on common raw starts. The four full seed-program medians are 921, 975, 992 and 1012 for Simulated Annealing, Tabu Search, Breakout Local Search and Burer–Monteiro respectively. Including the stronger relaxation and escape mechanism improves this initial population. No kernel definition or hidden work cap changes between the two scenarios.

More initial programs cost more numerical evaluations and alter the context supplied to Luna. Prior researcher design effort is also a real cost even though it is absent from the run clock. Compare

complete-program fitness, best-witness quality, model calls and numerical time. A fluent proposal, an incomplete high-scoring trial or a favorable single run does not establish a general advantage. The genuine replacement run made two model calls in 66.589 seconds. Burer–Monteiro followed by longer Breakout Local Search produced witnesses up to 1060, but its trials were incomplete, so the best eligible median stayed 1012. The second response referred to an ineligible parent and validation stopped the run. Replay labels this source FAILED. This is evidence of the validation and completeness rules, not a successful reference-matching run.

3.7 Comparing the evidence

If every method eventually reaches the same target, how should we compare them?
Compare initial fitness, complete-program improvement, reference gap and calls or time to a declared target. Retain unsuccessful and censored proposals.
Initial medians: **921 / 975 / 992 / 1012**. These distinguish the four configured solvers on the replacement graph; reference **1098** is unproven.
A common final score can hide different search costs. An unreached reference must remain visibly unreached.

The torus ceiling effect motivated a new instance screen rather than a fabricated failure. Five public graphs were screened using the same four initial configurations and seeds 3/4/5. The 1,000-vertex signed random graph gives a 91-cut spread between initial program medians, compared with two cut units on the old torus. The graph choice is informed by this numerical screen, so these three seeds are teaching evidence, not a held-out generalization test.

The two-call replacement results are: limited suite 975 $\rightarrow$ 1008 eligible median, best witness 1031, 24.003 seconds; informed suite 1012 $\rightarrow$ 1012 eligible median, best witness 1060, 66.589 seconds, terminated after an invalid parent reference. Neither matched 1098. The stronger starting population did not deliver the intended efficient-win narrative in these observations. Use the measured comparison table and genuine exact exchanges for actual model outcomes. Distinguish the primary median target from the stronger all-trial target and the separate best witness. A run that finishes its budget short of the reference still demonstrates mutation, evaluation and selection; it does not demonstrate successful reference attainment. Stored torus records remain labeled with their original graph and scenario, never substituted for replacement runs.

3.8 Demo 3: a dense real-weight stress case

Will the same architecture work when interactions are dense and weights are real-valued?
Use **100 vertices, 4950 real-valued signed edges**, all four initial solvers and supplied reference **402.5209**, unproven.
Eight numerical seconds per trial; three Luna calls maximum. Inspect composition, rejection, completeness and the actual surviving programs.
Evaluate the dense case on its own evidence. A reference match is a feasible result; it is not a proof of optimality or broad model superiority.

The dense Gaussian-weight graph comes from the supplied public assignment instances. Only graph input and scalar reference metadata were copied; no assignment solution vectors or private campaign records were imported. The dense case has 100 vertices and 4950 edges; the comparison graph has 1000 vertices and 5043 edges. Thus edge counts are similar while average degree, density

and weight distribution differ sharply. Both allow eight seconds per trial. This is a structural transfer case, not a justified universal ranking of instance difficulty.

The initial algorithms are the same disclosed configurations used elsewhere. All children share the same raw starts within this case. A previous two-call recording matched the reference with a Burer–Monteiro → Breakout Local Search composition and excluded a later time-limited extension. The current three-call rehearsal reached three complete reference matches after one model call at 16.091 seconds and finished at 56.602 seconds. It is recorded separately. Use the live comparison table and exact exchange to inspect its actual outcomes rather than substituting the old result.

3.9 Reading solver evidence

Which record proves that a solver improved: the graph, the hypothesis or the completed trials?
Require original-order integer signs and a matching graph hash. Recompute the objective from the original signed edges with two scoring paths.
Rank complete programs by median, minimum, fewer stages and stable genome ID. Retain a valid best witness separately, including one from a censored trial.
Selection follows complete measured evidence. Preserve failures, timeouts and plateaus so students can inspect the full mechanism.

A valid witness answers a feasibility-and-quality question for one execution. A complete candidate ranking answers how its program performed across the declared trials. Neither the model's hypothesis nor an attractive partition drawing supplies fitness. Incomplete programs receive no eligible median/minimum ranking entry, even if all returned witnesses happen to match the reference.

Changing work limits can trade quality against completion risk. The exact-exchange and comparison pages expose requested programs, measured execution time, actual token usage where supplied, invalid proposals and survivor decisions. Calls after a target has already been achieved are visible as teaching continuation. No extra activity or artificial solver motion is fabricated to fill the allotted demonstration window.

4 Formal definition and transfer

4.1 The agentic evolutionary framework

Can we now name the five choices we implemented?
$(\mathcal{A}, f, \mu, \sigma, P_0)$: artifact space, fitness, mutation, selection and initial population. Checkboxes choose P_0; they do not change $\mathcal{A}$.

$$Q_t \sim \mu(\cdot \mid P_t, H_t, B_t), \quad P_{t+1} = \sigma(P_t \cup E(Q_t)).$$

H_t: measured history. B_t: remaining budget. E: validation and complete evaluation.
The LLM is the mutation operator inside an externally measured evolutionary loop. For the course minimization convention, $f(a) = -\operatorname{median}_j C_G(A_a(G, z_j))$.

This instantiates Chapter 10, Definition 10.2, after the audience has encountered both ordinary evolutionary search and the LLM variant. The artifact space contains validated programs of one to three trusted stages. Its interpreter maps graph G and shared raw start z_j into signs under

the seeded numerical protocol. Negative median cut turns maximization into the course's lower-is-better fitness convention. The implementation supplements that primary scalar with minimum cut, stage count and stable identity tie-breaks.

The chapter presents an artifact-to-artifact mutation operator. The operational notation here makes context, stochasticity and budget explicit: one tool-free call may refer to up to three selected parents and return at most two children. Q_t is the proposed set; E(Q_t) retains only admitted, completely evaluated children. Selected programs and their measured histories inform the next request. The model's parameters remain fixed; learning during the run is carried by external context and population memory.

Elitism preserves observed measured strategies under this protocol, not global optimality, diversity or unseen-instance quality. Initial population design and allowed artifact-space design are separate levers. In this tutorial all checked and unchecked trusted kernels remain available to mutation in every comparison.

4.2 Applying the framework to research

Which component would be hardest to supply honestly for your own research problem? Specify the artifact, automatic evaluator, mutation context, survivor rule and initial population. Declare budgets and a success criterion before running.
Use domain knowledge, compare strong numerical baselines, and test unseen instances before claiming generalization.
Build trustworthy measurement and inspectable data flow first. Then decide where a language-model mutator earns its cost.

Transfer the framework by replacing components deliberately. A scientific expression may be scored by prediction error and complexity; an experimental protocol still needs a definition of executable evidence. If no reliable automatic evaluation exists, measurement design is the next task. The algorithm guide and separate architecture document let a researcher inspect the system before adapting it.

FunSearch supplies a prominent program-search example (Romera-Paredes et al. 2024). AlphaEvolve operates at a broader algorithm-evolution scope (AlphaEvolve team 2025), while OpenEvolve is an independent open-source implementation (OpenEvolve contributors n.d.). This classroom interpreter evolves configurations and compositions of existing kernels. It demonstrates the mechanism without claiming new kernel synthesis, broad model superiority or global convergence. The teaching sequence follows Chapter 10, especially Definition 10.2, Figures 10.1–10.2 and its discussions of context, elitism, diversity and cost.

A Appendix: architecture and API

A.1 Server architecture

What coordinates the browser, numerical search and model?

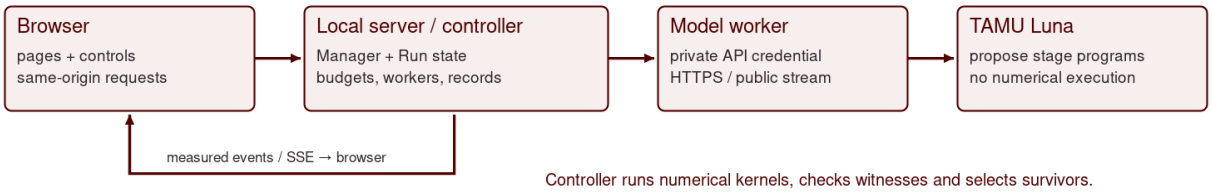


Figure 2: Only the model worker contacts TAMU.

One loopback server owns run state and workers, serves every teaching page, and streams events to the browser.
The server orchestrates; workers execute; events explain. The API key stays in the server-side client.

The diagram shows responsibilities, not a collection of independently deployed services. HTTP requests use a standard-library ThreadingHTTPServer on 127.0.0.1:8766. Manager permits one active run. A Run thread owns its sequence of serial numerical trials and model calls. Numerical and model workers are bounded child subprocesses with PID/start-tick ownership. Original-edge checking happens both inside the numerical adapter and again in the controller; the scorer box is not a separate network service.

The browser never directly calls TAMU. It creates a run, starts it, advances explicit round gates and subscribes to server-sent events. The same server hosts presentation, demo, guide, exact exchange, architecture and teaching notes. Switching views or reconnecting only reads existing state; it cannot start another paid request. The separate architecture document maps every route, module, message and trust boundary in more detail.

A.2 The API request and context

What is actually sent when we press "Send parents to Luna"?

```
model: protected.gpt-5.6-luna
messages:
  system: instructions + response schema
  user: serialized round context
stream: true; tools: []
```

Context: up to three genomes and trial evidence; failures and tried IDs; algorithm defaults, bounds and remaining budget. The worker adds HTTPS authorization separately.
The context carries measured search memory. Solution vectors, reference answers and the API key are excluded from the model's context.

At an explicit round gate, Run.propose selects the measured survivors and constructs a JSON context through proposals.context. Parent records contain normalized stages, median, minimum and three trial records, including elapsed numerical time, stage counters and completeness. The context also includes attempted genome identities, actual validation or incomplete-trial failures,

initial solver selection, graph size and signedness, the raw-start rule, allowed defaults, parameter bounds, stage work caps and remaining wall time.

The model receives two Chat Completions messages: a system message with instructions plus the required JSON schema, and a user message containing that context. The TAMU client sends a streamed HTTPS POST to https://chat-api.tamu.ai/api/chat/completions with the protected Luna model, max_completion_tokens=2400, low reasoning effort and no tools. Authorization is a server-side header, not part of the displayed body. The proposer is not given the assignment vectors, stored numerical preset, replay, benchmark name or reference target. The exact-exchange page displays the actual recorded request body and public return, rather than a reconstructed example.

A.3 Response and loop closure

When does streamed text become a candidate that can survive?
Public chunks → completed JSON → known parent IDs → schema, bounds and duplicate checks → admitted stage program.
Owned numerical trials → independent original-edge scores → complete-candidate ranking → new survivors and failure feedback for the next request.
Text is a proposal until validated and measured. Server-sent events expose progress; selection supplies the next mutation context.

model_worker.py projects public content and completion metadata from the provider stream, excluding hidden reasoning fields and authorization. A watcher emits actual public deltas to the browser. A completed response must be valid JSON with the required round ID, known parent IDs, hypothesis, mutation category, up to two children and any plateau reason. Each child is checked independently; invalid parameters and duplicate normalized genomes are rejected without a silent fix or free retry.

Admitted children run through backend.py in owned processes. The controller checks their graph hash and independently rescored witnesses. Incomplete candidates cannot enter the survivor ranking, even when their best witness is valid. Each event is appended to an ignored run log and streamed by SSE with a sequence number. Reconnection uses an event cursor and never launches paid work. A genuine recorded replay reproduces these events, keeps a persistent replay label, and reveals its public response only when the recorded timeline reaches that response.

References

AlphaEvolve team. 2025. "AlphaEvolve: A Gemini-Powered Coding Agent for Designing Advanced Algorithms." Google DeepMind. May 14, 2025. https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/.

OpenEvolve contributors. n.d. "OpenEvolve: Independent Open-Source Implementation of AlphaEvolve." Accessed October 26, 2026. https://github.com/algorithmicsuperintelligence/openevolve.

Romera-Paredes, Bernardino, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, et al. 2024. "Mathematical Discoveries from Program Search with Large Language Models." *Nature* 625: 468–75. https://doi.org/10.1038/s41586-023-06924-6.